

# **Course Offering Recommendation System (CORS)**

## **CSC 599 – Capstone Project**

### **Final Report**

**Department of Mathematics and Computer Science**

**Lebanese American University (LAU) - Byblos Campus**

**Ayman Kacan & Tarek Al Saleh**

**Dr. Danielle Azar**

**Spring 2026**

## **Introduction**

The scheduling of academic courses and the allocation of educational resources every semester are critical, highly complex processes for any university department. This capstone project focuses on the development of a Course Offering Recommendation System (CORS) specifically designed for the Computer Science and Mathematics Department at the Lebanese American University (LAU). The primary objective of this system is to generate an optimized list of courses and calculate the precise number of sections needed for each course in the upcoming semester or academic year.

Traditionally, academic assistants and department chairs must manually sift through convoluted data to make scheduling decisions. CORS serves to streamline this by acting as an AI-powered academic planning platform built for university administrators. By analyzing the institution's historical enrollment data, CORS applies trained machine learning models to recommend which courses to offer and how many sections to open in order to schedule them effectively. The output is intended to assist the department chairperson by providing a data-informed reference list. This generated list can then be considered alongside other operational factors, such as faculty availability and classroom capacities, to create a finalized schedule.

While the system was initiated for the Byblos campus, the resulting architecture supports both LAU campuses (Beirut and Byblos), utilizing independent machine learning models trained on each campus's distinct enrollment history. This ensures that recommendations always reflect the correct institutional context and campus-specific demand patterns.

CORS is built for department chairs and assistants who are responsible for building the semester's course schedule. The platform consolidates four core capabilities into a single workflow:

- **AI Course Recommendations:** Machine-learning models evaluate the course catalog to recommend whether a course should be offered (labeled as "Recommended" or "De-prioritized") and provide a projected section count based on projected demand and historical metrics.
- **Prerequisite Graphing:** The system features an interactive visualization of course dependencies across the curriculum, allowing administrators to identify bottleneck courses that block large numbers of students from progressing.
- **Timetable Scheduling:** Once the course list is determined, administrators utilize a drag-and-drop scheduler to assign time slots, rooms, and faculty, with the system automatically detecting real-time time conflicts.
- **Data Management:** A centralized hub allows for the upload and management of the course catalog, historical offering records, and faculty rosters, forming the data foundation that powers the predictive models.

The objective of this project is to integrate a mechanism to support the chairperson in identifying course sequences and avoiding delays in graduation through intelligent predictions and alerts. By bridging the gap between raw institutional data and actionable scheduling interfaces, CORS aims to transform course planning from a manual, intuitive task into a data-driven, risk-aware and calculated process.

## **User Requirements**

The system's requirements have been carefully defined to address the functional needs of the department chairperson within the project scope. These requirements govern the data inputs, the analytical processing, and the user interface capabilities.

### System Inputs and Data Ingestion Requirements

To produce accurate recommendations, the system requires a robust foundation of institutional data. The system must ingest and process the following inputs:

- Student academic records with complete anonymity ensured, encompassing grades, enrollments, derived expected graduation timelines, and pass/fail averages.
- Study plans and overarching curriculum requirements.
- Course relations and course tree data, specifically prerequisites and co-requisites.
- Historical enrollment data and demand trends.
- The course catalog must be uploaded as a JSON file, defining every course, its prerequisites, its classification (e.g., core, elective, masters), and its associated study plan.
- Historical offerings must be uploaded in CSV or XLSX formats, detailing past semester enrollment records, including the total students enrolled, the number of students who passed, and the number of students who failed.
- Faculty records must be ingested (via JSON, CSV, or XLSX) to define professor names, course qualifications, and specific availability windows (e.g., "MWF 08:00 12:00").

## Functional Requirements

The core utility of CORS lies in its functional capabilities, which empower the chair to make informed, data-backed decisions.

### Course and Section Recommendations

- The system must generate a list of recommended courses for the coming semester.
- The system must suggest the precise number of sections required for each individual course.
- The system must calculate a "Projected sections" metric by dividing latent demand by the typical section capacity for that course level.
- The recommendation output must provide a binary status of either "Recommended" or "De-prioritized" based on a machine learning probability threshold.
- The system must allow users to enforce "Strict Capacity Quotas," which constrains the recommendation output to a fixed number of course slots per department and course type.

### Interface, Dashboard, and Filtering

- The platform must feature a proper user interface and dashboard equipped with filters by program, course type (such as core or elective), and urgency (highlighting graduation-critical courses through latent demand and bottleneck scores).
- The dashboard must display high-level KPI cards, including the total number of courses in the catalog, total processed historical enrollment records, the machine-

learning model's ROC-AUC score, and the historical average failure rate across all offerings.

- Users must be able to filter the recommendation results table by department (CSC, BIF, MTH, and STA), academic category, offer status, and demand index.

### Risk Analysis and Consequence Forecasting

- The system must perform a risk analysis for each course offering and planning decision.
- The system must describe and notify the user of the consequences of certain choices in course offerings or withholdings.
- To fulfill this, the system must provide an AI reasoning popover (accessible via a "Why?" button) that explains the model's decision by displaying the course's Latent Demand, Bottleneck Score, and Model Confidence percentage.
- The system must calculate an Impact Score using the formula:  $\text{Latent Demand} + (5.0 \times \text{Bottleneck Score})$ .
- Any course with an Impact Score exceeding 150 must be dynamically flagged as a "systemic bottleneck" within the user interface, generating a red alert banner to notify administrators that capacity expansion is mandatory for degree progression.

### Tentative Scheduling and Conflict Resolution

- A timetable-like interface for tentative scheduling must be included for practicality.

- The timetable orchestration module must allow users to translate recommendations into an actual semester schedule by assigning courses to weekly time slots, faculty, and rooms.
- The scheduler must support day pattern notation utilizing single-letter abbreviations (e.g., M, T, W, R, and F) combined into compact strings like "MWF" or "TR".
- The system must provide automatic conflict detection in real time, specifically enforcing professor availability constraints, preventing professor double-booking, and preventing room occupancy conflicts.

#### Export Capabilities

- The system must allow the user to finalize and export the complete semester schedule as a Microsoft Excel workbook (XLSX).
- The exported schedule file must include the days pattern, course code, section name, instructor, start time, end time, and assigned room.

#### Admin, Logging and User Management

- The system must support an admin user and multiple users within the department.
- The Admin user must be able to grant access to users through the dashboard, with authentication and authorization functionalities in place.
- The Admin must have access to system logs, model retraining, database and user management, while users can only access trained models and scheduling.
- Only the Admin has access to data imports and removal, users interact with state of data and models as is, as decided by Admin actions.

Database Diagram

| courses |               |    |
|---------|---------------|----|
| String  | code          | PK |
| String  | name          |    |
| String  | prefix        |    |
| String  | number        |    |
| String  | type          |    |
| String  | study_plan    |    |
| Text    | prerequisites |    |
| Boolean | is_math       |    |
| Boolean | is_core       |    |
| Integer | course_level  |    |
| Text    | aliases       |    |

| course_offerings |                |    |
|------------------|----------------|----|
| Integer          | id             | PK |
| Integer          | year           |    |
| String           | semester       |    |
| String           | campus         |    |
| String           | course_code    |    |
| Integer          | total_enrolled |    |
| Integer          | passed_count   |    |
| Integer          | failed_count   |    |
| Float            | fail_ratio     |    |
| Boolean          | is_offered     |    |
| DateTime         | created_at     |    |

| data_audit_logs |                |    |
|-----------------|----------------|----|
| Integer         | id             | PK |
| String          | table_affected |    |
| String          | action         |    |
| String          | description    |    |
| String          | username       |    |
| DateTime        | created_at     |    |

| action_logs |             |    |
|-------------|-------------|----|
| Integer     | id          | PK |
| String      | username    |    |
| String      | action      |    |
| String      | description |    |
| DateTime    | created_at  |    |

| prediction_runs |                 |    |
|-----------------|-----------------|----|
| Integer         | id              | PK |
| String          | run_name        |    |
| String          | campus          |    |
| Integer         | target_year     |    |
| String          | target_semester |    |
| String          | model_version   |    |
| DateTime        | created_at      |    |

| doctors |                 |    |
|---------|-----------------|----|
| Integer | id              | PK |
| String  | name            |    |
| Text    | allowed_courses |    |

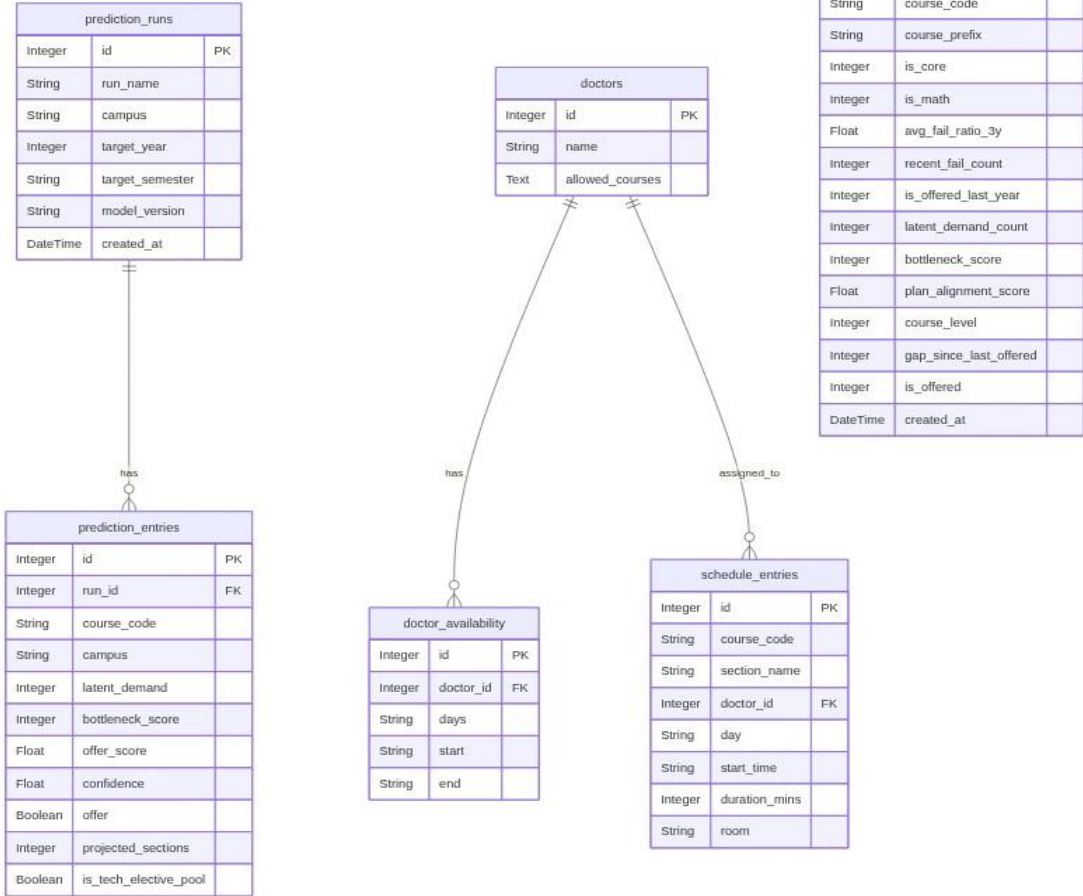
| training_records |                        |    |
|------------------|------------------------|----|
| Integer          | id                     | PK |
| Integer          | year                   |    |
| String           | semester               |    |
| String           | campus                 |    |
| String           | course_code            |    |
| String           | course_prefix          |    |
| Integer          | is_core                |    |
| Integer          | is_math                |    |
| Float            | avg_fail_ratio_3y      |    |
| Integer          | recent_fail_count      |    |
| Integer          | is_offered_last_year   |    |
| Integer          | latent_demand_count    |    |
| Integer          | bottleneck_score       |    |
| Float            | plan_alignment_score   |    |
| Integer          | course_level           |    |
| Integer          | gap_since_last_offered |    |
| Integer          | is_offered             |    |
| DateTime         | created_at             |    |

| users   |               |    |
|---------|---------------|----|
| Integer | id            | PK |
| String  | username      |    |
| String  | password_hash |    |
| String  | email         |    |
| Boolean | is_active     |    |
| Boolean | is_admin      |    |

| prediction_entries |                       |    |
|--------------------|-----------------------|----|
| Integer            | id                    | PK |
| Integer            | run_id                | FK |
| String             | course_code           |    |
| String             | campus                |    |
| Integer            | latent_demand         |    |
| Integer            | bottleneck_score      |    |
| Float              | offer_score           |    |
| Float              | confidence            |    |
| Boolean            | offer                 |    |
| Integer            | projected_sections    |    |
| Boolean            | is_tech_elective_pool |    |

| doctor_availability |           |    |
|---------------------|-----------|----|
| Integer             | id        | PK |
| Integer             | doctor_id | FK |
| String              | days      |    |
| String              | start     |    |
| String              | end       |    |

| schedule_entries |               |    |
|------------------|---------------|----|
| Integer          | id            | PK |
| String           | course_code   |    |
| String           | section_name  |    |
| Integer          | doctor_id     | FK |
| String           | day           |    |
| String           | start_time    |    |
| Integer          | duration_mins |    |
| String           | room          |    |



## Technologies Used

The underlying technology stack of the Course Offering Recommendation System (CORS) integrates modern predictive modeling, a robust RESTful architecture, efficient frontend components, an efficient backend structure, rigorous security protocols and deployment flows. The system relies heavily on specific industry-standard languages and libraries to achieve these functional requirements.

### Machine Learning Component

The system's core intelligence is driven by an advanced recommendation engine that utilizes customized predictive modeling. The application employs campus-specific ensemble machine learning models, ensuring that the Beirut and Byblos campuses have independent predictive weights calibrated to their respective enrollment histories. The models evaluate course viability by extracting feature vectors that include historical failure rates, enrollment trends, and projected student influx. In standard operational mode, the models apply an F1.5-score optimized classification threshold to decide whether a course should be recommended. Model health and predictive discrimination power are quantified using the Area Under the ROC Curve (Model ROC-AUC), a metric prominently displayed on the system dashboard.

### Technologies utilized for this module include:

- Languages: Python
- Libraries: Scikit-learn and XGBoost (for constructing and training the ensemble classification models), Pandas and NumPy (for extracting feature vectors, manipulating historical enrollment datasets, and calculating latent demand),

Matplotlib (for visualization during data analysis), Joblib (pickling of models and ML exports)

- Tools: Google Collab Notebooks (utilized during the development phase for model training, tuning the F1.5-score threshold and evaluating ROC-AUC metrics)

### Frontend Components

The frontend provides the dynamic, interactive user interface for the CORS platform, enabling university administrators to seamlessly interact with the underlying data and predictive models. It acts as the presentation layer, managing application state and client-side routing to navigate fluidly between various administrative dashboards, including course management, system audit logs, and user authorization panels. The frontend is responsible for making asynchronous HTTP requests to the backend API to execute CRUD operations and fetch processed data. Crucially, it translates the complex, raw output generated by the machine-learning pipeline into intuitive data visualizations, such as interactive charts for threshold metrics and node-based graphs for course prerequisite structures, ensuring that the analytical insights are both highly accessible and actionable.

### Technologies utilized for this module include:

- Languages: TypeScript, JavaScript, HTML5, CSS3
- Core Frameworks & Build Tools:
  - React (v18): The foundational library used for building component-driven, reactive user interfaces.
  - Vite: A modern, lightning-fast build tool and development server that significantly optimizes the frontend compilation process.

➤ Routing & Networking Libraries:

- React Router DOM: Manages complex client-side routing and navigation between different application views without requiring page reloads.
- Axios: A promise-based HTTP client used to seamlessly communicate with the FastAPI backend.

➤ Data Visualization Libraries:

- Recharts: A composable charting library built on React components, used to display analytical metrics and prediction thresholds.
- React Flow & Dagre: Specialized libraries used to render and auto-layout complex node-based graphs, specifically for visualizing course prerequisites and co-requisites.

➤ Styling & UI Assets:

- Tailwind CSS: A utility-first CSS framework utilized for rapid, responsive, and highly customizable UI styling.
- Lucide-React: Provides a comprehensive suite of modern, consistent SVG icons for the interface.

## Backend Structure & RESTful Architecture

The backend serves as the core operational layer of the CORS (Course Offering Recommendation System) platform, acting as the intelligent bridge between the user interface, the underlying database, and the predictive models. It is heavily responsible for database management and executing comprehensive CRUD (Create, Read, Update, Delete) operations, allowing administrators to seamlessly interact with RESTful API endpoints to manage user accounts, course catalogs, historical records, and system audit logs. Furthermore, the backend drives the system's data extraction and analysis capabilities; it systematically queries raw data from the PostgreSQL database, performs rigorous data cleaning and feature engineering using Pandas, and feeds this structured data into the dual-campus machine learning pipeline for Beirut and Byblos. Beyond data handling, the backend orchestrates critical automated workflows such as model training and predictive scheduling, while strictly enforcing security through JWT-based authentication, role-based access control (RBAC), and automated email notifications.

### Technologies utilized for this module include:

- Languages: Python
- Frameworks:
  - FastAPI: For building high-performance, asynchronous RESTful APIs.
  - Uvicorn: The ASGI web server implementation used to run the FastAPI application.
- Machine Learning & Data Processing Libraries:
  - Pandas: For robust data manipulation and cleaning.

- Scikit-learn & XGBoost: For building, training, and evaluating the predictive ensemble models.
- Joblib: For serializing and saving the trained machine learning pipelines.

➤ Database & Security Libraries:

- SQLAlchemy: As the Object-Relational Mapper (ORM) to manage database transactions and schema modeling.
- Pydantic: For strict data validation and environment variable management.
- PyJWT & Passlib (Bcrypt): For secure generation of JSON Web Tokens and password hashing.

➤ Tools & Infrastructure:

- PostgreSQL: The primary relational database management system (connected via psycopg2).
- SendGrid: For integrating and sending automated email notifications.
- Docker: Used to containerize the backend environment to ensure consistency across deployment stages.

## Security Protocols

Given the requirement to strictly protect sensitive student and institutional data, the CORS platform enforces rigorous authentication methodologies. The system utilizes a robust two-step authentication flow to safeguard administrative access. Users must first submit their credentials to the /auth/login endpoint, which subsequently dispatches a single-use, 6-digit verification code directly to the user's registered academic email address. Upon successful Two-Factor Authentication (2FA) verification, the backend API issues a cryptographically signed JSON Web Token (JWT). To balance usability with strict security standards, these tokens are configured with a one-week expiration window, requiring administrators to re-authenticate periodically to maintain access. The platform relies on Bearer token authentication, requiring this JWT to be passed in the Authorization header of all protected API requests to seamlessly validate and authorize access.

### Technologies utilized for this module include:

- Languages: Python (Backend) / JavaScript & TypeScript (Frontend)
- Libraries & Frameworks: PyJWT (for secure token generation and validation), passlib using PBKDF2 SHA256 (for cryptographic password hashing), and sendgrid (for email delivery).
- Tools: SMTP Integration (utilized for reliably dispatching the single-use 6-digit 2FA codes to academic email addresses).

## Deployment and Infrastructure Stack

This is how the system is hosted, containerized, and served in production.

- Docker & Docker Compose: Used to containerize the entire application (frontend, backend, and database), ensuring consistent environments across local development and production. Docker Compose orchestrates the multi-container setup.
- Nginx: Serves as a high-performance web server and reverse proxy within the Dockerized frontend container, efficiently serving the compiled Vite/React static assets.
- Vercel: The cloud platform used for hosting and deploying the frontend application, providing fast global content delivery and continuous integration.
- Render: The cloud platform utilized for deploying the FastAPI backend and managing the production PostgreSQL database.
- Git / GitHub: Used for version control, tracking changes, and enabling collaborative development and deployment workflows.

## **Links**

<https://cors-lau.vercel.app/> (deployed website)

<https://cors-lau.vercel.app/docs> (interactive documentation web page)

<https://github.com/Tarek-AL-Saleh/CORS> (GitHub repo of the project)